

ROTEIRO DE AULA

DDL x DML E COMANDOS DML AVANÇADOS

Inserção, atualização, exclusão em tabelas relacionadas e triggers em SQL
Com exemplos teóricos, analogias, prática em MySQL e alternativa no OneCompiler/SQLite

Observação ao professor: Este roteiro foi ampliado com exemplos na parte teórica para que o aluno compreenda a lógica antes de executar os comandos. A proposta é evitar que a prática seja apenas cópia mecânica de SQL.

Revisão técnica dos códigos: A versão OneCompiler/SQLite foi revisada e testada em sequência, com funcionamento correto até o teste final de bloqueio, que deve gerar erro de propósito. A versão MySQL foi revisada e reorganizada para deixar o teste de erro por último, evitando interromper etapas posteriores.

Materiais de referência: Aula 1 - Inserção de dados em tabelas relacionadas; Aula 2 - Atualização de dados em tabelas relacionadas; Aula 3 - Exclusão de dados em tabelas relacionadas; Aula 4 - Trigger e automação de tarefas; Atividade prática - Automação de tarefas com triggers em SQL.

1. Tema da aula

DDL x DML e comandos DML avançados aplicados a tabelas relacionadas, com foco em INSERT, UPDATE, DELETE, JOIN, GROUP BY, HAVING, NOT EXISTS e TRIGGER.

2. Objetivo geral

Compreender como bancos de dados relacionais armazenam informações em tabelas conectadas e como os comandos SQL podem inserir, atualizar, excluir e automatizar ações de forma segura e organizada.

3. Habilidades trabalhadas

- Diferenciar comandos DDL e DML.
- Interpretar tabelas relacionadas por chave primária e chave estrangeira.
- Utilizar comandos INSERT, UPDATE, DELETE e SELECT em cenários simples.
- Compreender JOIN como forma de conectar informações entre tabelas.
- Criar e testar triggers simples para automação, auditoria e proteção de dados.
- Registrar evidências da prática em Word/Docs e Excel/Sheets.
- Analisar erros comuns sem desespero, entendendo a causa provável.

4. Conteúdos principais

Conteúdo	Ideia central	Analogia rápida
Banco de dados relacional	Organiza informações em tabelas que se relacionam.	Um armário com várias gavetas organizadas.
Tabela	Estrutura onde os dados são guardados em linhas e colunas.	Uma planilha do Excel.
Chave primária	Identifica cada registro de forma única.	RG/CPF de uma pessoa.
Chave estrangeira	Liga uma tabela a outra.	Uma ponte entre duas fichas.
DDL	Comandos que criam ou alteram a estrutura.	Construir ou reformar a casa.

DML	Comandos que manipulam os dados.	Usar a casa depois de pronta.
JOIN	Junta informações de tabelas relacionadas.	Juntar dados de gavetas diferentes.
Trigger	Ação automática disparada por um evento.	Alarme que dispara sozinho.

5. Aula teórica - conceitos com analogias e exemplos

Estratégia didática: Para cada conceito, apresente primeiro a analogia, depois o exemplo do cotidiano e só então o comando SQL. Isso ajuda o aluno iniciante a entender o porquê antes do código.

5.1 Banco de dados relacional

Definição: Um banco de dados relacional guarda informações em tabelas que podem se conectar entre si. Em vez de colocar tudo em uma única tabela gigante, separamos as informações por assunto.

Analogia: Imagine um armário de secretaria escolar: uma gaveta guarda fichas de alunos, outra guarda matrículas, outra guarda notas. As gavetas são separadas, mas se conectam pelo número do aluno.

Exemplo de uso: Em uma loja, podemos ter uma tabela de clientes e uma tabela de pedidos. O cliente João fica registrado uma vez na tabela clientes. Cada compra dele fica registrada na tabela pedidos, ligada ao código do João.

```
-- Exemplo conceitual de tabelas relacionadas
clientes: id_cliente, nome, email
pedidos: id_pedido, id_cliente, data_pedido, status
```

5.2 Tabela, campo e registro

Definição: Tabela é a estrutura que armazena dados. Campo é a coluna. Registro é a linha com os dados de uma entidade.

Analogia: Pense em uma planilha: as colunas são campos, e cada linha preenchida é um registro.

Exemplo de uso: Na tabela clientes, os campos são id_cliente, nome e email. Um registro seria: 1, Ana Souza, ana@email.com.

```
-- Visualizando a ideia
id_cliente | nome      | email
1          | Ana Souza | ana@email.com
2          | Bruno    | bruno@email.com
```

5.3 Chave primária

Definição: A chave primária identifica cada linha de uma tabela sem repetição. Ela evita confusão entre registros parecidos.

Analogia: É como o RG ou CPF: duas pessoas podem ter o mesmo nome, mas não o mesmo CPF.

Exemplo de uso: Dois clientes podem se chamar João, mas cada um terá um id_cliente diferente.

```
CREATE TABLE clientes (
  id_cliente INT PRIMARY KEY AUTO_INCREMENT,
  nome VARCHAR(100),
  email VARCHAR(100)
);
```

5.4 Chave estrangeira

Definição: A chave estrangeira é um campo que cria ligação entre duas tabelas. Ela aponta para a chave primária de outra tabela.

Analogia: É como escrever na ficha de um pedido o número do cliente que fez a compra. O pedido não precisa repetir todos os dados do cliente; basta guardar o código que liga ao cliente correto.

Exemplo de uso: Na tabela pedidos, o campo id_cliente indica qual cliente fez cada pedido.

```
CREATE TABLE pedidos (
  id_pedido INT PRIMARY KEY AUTO_INCREMENT,
  id_cliente INT,
  data_pedido DATE,
  status VARCHAR(30),
  FOREIGN KEY (id_cliente) REFERENCES clientes(id_cliente)
);
```

5.5 DDL x DML

Definição: DDL cria e altera a estrutura do banco. DML manipula os dados que ficam dentro dessa estrutura.

Analogia: DDL é construir a casa: parede, cômodo, porta. DML é usar a casa: entrar, trocar móveis, tirar objetos.

Exemplo de uso: CREATE TABLE é DDL porque cria uma tabela. INSERT é DML porque coloca dados dentro da tabela.

```
-- DDL: cria estrutura
CREATE TABLE produtos (
  id_produto INT PRIMARY KEY AUTO_INCREMENT,
  nome VARCHAR(100)
);

-- DML: manipula dados
INSERT INTO produtos (nome) VALUES ('Mouse Gamer');
```

5.6 INSERT - inserindo dados

Definição: INSERT é usado para cadastrar novos registros em uma tabela.

Analogia: É como preencher uma nova ficha de cadastro e guardar no arquivo.

Exemplo de uso: Cadastrar uma cliente chamada Ana na tabela clientes.

```
INSERT INTO clientes (nome, email, data_cadastro)
VALUES ('Ana Souza', 'ana@datacenter.com', CURDATE());
```

5.7 Inserção em tabelas relacionadas

Definição: Quando duas tabelas são relacionadas, normalmente cadastramos primeiro a tabela principal e depois a tabela dependente.

Analogia: Antes de registrar uma compra, a loja precisa saber quem é o cliente. Primeiro cria a ficha do cliente, depois registra o pedido ligado a essa ficha.

Exemplo de uso: Cadastrar Ana em clientes e depois criar um pedido ligado ao id_cliente dela.

```
INSERT INTO clientes (nome, email, data_cadastro)
VALUES ('Ana Souza', 'ana@datacenter.com', CURDATE());
```

```
INSERT INTO pedidos (id_cliente, data_pedido, status)
VALUES (1, CURDATE(), 'Pendente');
```

5.8 JOIN - juntando informações

Definição: JOIN permite combinar informações de duas ou mais tabelas relacionadas.

Analogia: É como abrir duas gavetas ao mesmo tempo: a gaveta dos clientes e a gaveta dos pedidos, juntando o pedido com o dono correto.

Exemplo de uso: Mostrar o nome do cliente junto com o status do pedido.

```
SELECT c.nome, p.id_pedido, p.status
FROM clientes AS c
JOIN pedidos AS p
ON c.id_cliente = p.id_cliente;
```

5.9 INSERT INTO ... SELECT ... JOIN

Definição: Esse comando insere dados em uma tabela usando o resultado de uma consulta que junta outras tabelas.

Analogia: Imagine que a escola quer criar uma lista de alunos premiados. Em vez de digitar tudo manualmente, o sistema pega alunos que cumprem uma regra e joga automaticamente na lista.

Exemplo de uso: Inserir em clientes_premiados os clientes online que fizeram pelo menos três pedidos.

```
INSERT INTO clientes_premiados (id_cliente, nome, data_premiacao)
SELECT c.id_cliente, c.nome, NOW()
FROM clientes AS c
JOIN pedidos AS p ON c.id_cliente = p.id_cliente
WHERE c.tipo_cliente = 'online'
GROUP BY c.id_cliente, c.nome
HAVING COUNT(p.id_pedido) >= 3;
```

5.10 NOT EXISTS - evitando duplicidade

Definição: NOT EXISTS verifica se um registro ainda não existe antes de inserir. Isso evita duplicar dados.

Analogia: É como conferir se o nome do aluno já está na lista antes de escrevê-lo novamente.

Exemplo de uso: Antes de inserir um cliente na tabela de premiados, verificar se ele já não foi premiado.

```
INSERT INTO clientes_premiados (id_cliente, nome, data_premiacao)
SELECT c.id_cliente, c.nome, NOW()
FROM clientes AS c
WHERE NOT EXISTS (
    SELECT 1
    FROM clientes_premiados AS cp
    WHERE cp.id_cliente = c.id_cliente
);
```

5.11 GROUP BY e HAVING

Definição: GROUP BY agrupa registros. HAVING filtra os grupos depois da contagem ou soma.

Analogia: GROUP BY é separar alunos por turma. HAVING é escolher apenas as turmas que têm mais de 30 alunos.

Exemplo de uso: Contar pedidos por cliente e mostrar apenas quem fez três ou mais pedidos.

```
SELECT c.nome, COUNT(p.id_pedido) AS total_pedidos
FROM clientes AS c
JOIN pedidos AS p ON c.id_cliente = p.id_cliente
GROUP BY c.id_cliente, c.nome
HAVING COUNT(p.id_pedido) >= 3;
```

5.12 UPDATE JOIN - atualizando dados relacionados

Definição: UPDATE JOIN altera registros de uma tabela usando informações de outra tabela relacionada.

Analogia: É como atualizar a data de entrega de todos os projetos de funcionários de um departamento. Você consulta a ficha do funcionário para decidir quais projetos serão alterados.

Exemplo de uso: Atualizar o status de pedidos atrasados apenas para clientes ativos.

```
UPDATE pedidos AS p
JOIN clientes AS c ON p.id_cliente = c.id_cliente
SET p.status = 'Atrasado'
WHERE p.data_pedido < CURDATE() - INTERVAL 7 DAY
AND c.status_cliente = 'Ativo';
```

5.13 DELETE JOIN - excluindo dados relacionados

Definição: DELETE JOIN exclui registros de uma tabela com base em uma relação com outra tabela.

Analogia: É como apagar fichas de pedidos antigos consultando primeiro a ficha do cliente ou a data do pedido.

Exemplo de uso: Excluir pedidos feitos há mais de dois anos.

```
DELETE p
FROM pedidos AS p
JOIN clientes AS c ON p.id_cliente = c.id_cliente
WHERE p.data_pedido < CURDATE() - INTERVAL 2 YEAR;
```

5.14 Trigger - automação no banco

Definição: Trigger é uma rotina automática que o banco executa quando acontece um evento, como inserir, atualizar ou excluir um registro.

Analogia: É como um alarme de loja: quando alguém abre a porta, o alarme dispara sozinho. Você não aperta o botão toda vez.

Exemplo de uso: Quando o status de um pedido muda, a trigger registra a data da alteração em uma tabela de auditoria.

```
CREATE TRIGGER after_pedido_update
AFTER UPDATE ON pedidos
FOR EACH ROW
BEGIN
    INSERT INTO auditoria_pedidos (id_pedido, data_atualizacao)
    VALUES (NEW.id_pedido, NOW());
END;
```

5.15 Auditoria e prevenção de exclusões indevidas

Definição: Auditoria registra o que aconteceu. Prevenção impede que uma ação perigosa seja realizada.

Analogia: Auditoria é uma câmera de segurança. Prevenção é uma trava na porta que impede alguém de entrar sem autorização.

Exemplo de uso: Bloquear a exclusão de pedidos com status Enviado.

```

CREATE TRIGGER before_pedido_delete
BEFORE DELETE ON pedidos
FOR EACH ROW
BEGIN
    IF OLD.status = 'Enviado' THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Não é permitido excluir pedidos enviados.';
    END IF;
END;

```

6. Roteiro da aula teórica - 50 minutos

Tempo	Momento	Condução sugerida
0-5 min	Abertura	Apresente a pergunta: se uma loja tem clientes e pedidos, por que não colocar tudo em uma tabela só?
5-15 min	Banco relacional, tabelas e chaves	Use a analogia do armário, gavetas, fichas e RG/CPF para explicar tabela, registro, campo, chave primária e chave estrangeira.
15-22 min	DDL x DML	Explique DDL como construir a casa e DML como usar a casa. Mostre exemplos de CREATE TABLE e INSERT.
22-32 min	INSERT, JOIN, NOT EXISTS, GROUP BY e HAVING	Mostre exemplos de inserção em tabela relacionada e de seleção de clientes com três ou mais pedidos.
32-40 min	UPDATE JOIN e DELETE JOIN	Explique atualização e exclusão em tabelas relacionadas, reforçando cuidados para não alterar ou apagar dados errados.
40-47 min	Triggers	Explique trigger como alarme automático. Mostre exemplos de auditoria e bloqueio de exclusão.
47-50 min	Fechamento teórico	Peça para os alunos responderem: qual comando cria estrutura? qual manipula dados? o que uma trigger automatiza?

7. Aula prática - 50 minutos

Propósito da prática: Criar uma base de dados simples da empresa fictícia DataSoft e testar triggers que automatizam tarefas, registram auditoria e protegem pedidos importantes.

Tempo	Atividade	O que o aluno deve entender
0-5 min	Apresentação da DataSoft	O banco representa uma empresa que possui clientes e pedidos.
5-12 min	Criar banco e tabelas	DDL cria a estrutura, como montar as gavetas antes de guardar fichas.
12-18 min	Inserir dados iniciais	DML preenche as fichas para testar o sistema.
18-25 min	Consultar dados	SELECT é olhar o que existe dentro das tabelas.
25-35 min	Criar triggers principais	Triggers automatizam ações que antes seriam manuais.
35-45 min	Testar triggers	Observar antes e depois para provar que a automação funcionou.
45-50 min	Organizar evidências	Salvar prints no Word/Docs e preencher tabela no

		Excel/Sheets.
--	--	---------------

8. Materiais necessários

- Computador com acesso à internet.
- Ferramenta para executar SQL MySQL, como MySQL Workbench, XAMPP/phpMyAdmin, DB Fiddle ou ambiente equivalente.
- Opcional para quem não consegue usar MySQL na escola: OneCompiler no navegador, selecionando a linguagem SQLite.
- Word ou Google Docs para organizar prints.
- Excel ou Google Sheets para tabela de acompanhamento.
- Caderno para anotações e rascunhos.

Importante: A prática principal continua em MySQL, pois os materiais de referência usam exemplos como NOW(), CURDATE(), INTERVAL e UPDATE JOIN. Porém, como algumas máquinas da escola podem não permitir MySQL, há uma versão alternativa para OneCompiler usando SQLite. Quem tiver MySQL pode seguir a versão MySQL; quem não tiver, usa o OneCompiler.

9. Atividade prática guiada - DataSoft

Contexto: você trabalha como desenvolvedor de banco de dados na empresa fictícia DataSoft. A empresa precisa automatizar tarefas para reduzir erros manuais no controle de clientes e pedidos.

Etapa 1 - Criar o banco de dados

O que será feito: criar uma pasta principal chamada datassoft onde as tabelas da empresa ficarão guardadas.

Analogia: criar o banco é como abrir uma pasta nova no computador chamada DataSoft. Dentro dela ficarão as planilhas clientes, pedidos, auditorias e alertas.

```
DROP DATABASE IF EXISTS datassoft;
CREATE DATABASE datassoft;
USE datassoft;
```

Explicação: DROP DATABASE IF EXISTS remove uma versão antiga para recomençar limpo; CREATE DATABASE cria o banco; USE escolhe esse banco para os próximos comandos.

Print: tire print mostrando os comandos executados sem erro.

Etapa 2 - Criar as tabelas

O que será feito: criar as gavetas do armário da empresa. Cada tabela guarda um tipo de informação.

Analogia: clientes é a gaveta das fichas de clientes; pedidos é a gaveta das compras; auditoria é a câmera de segurança; alertas é o quadro de avisos.

```
CREATE TABLE clientes (
  id_cliente INT PRIMARY KEY AUTO_INCREMENT,
  nome VARCHAR(100) NOT NULL,
  email VARCHAR(120) NOT NULL UNIQUE,
  status_cliente VARCHAR(20) DEFAULT 'Ativo',
  tipo_cliente VARCHAR(20) DEFAULT 'online',
  data_cadastro DATE DEFAULT (CURRENT_DATE)
);
```

```
CREATE TABLE pedidos (
  id_pedido INT PRIMARY KEY AUTO_INCREMENT,
  id_cliente INT NOT NULL,
  data_pedido DATE NOT NULL,
  status VARCHAR(30) NOT NULL,
  FOREIGN KEY (id_cliente) REFERENCES clientes(id_cliente)
);
```

```
CREATE TABLE auditoria_pedidos (
  id_auditoria INT PRIMARY KEY AUTO_INCREMENT,
  id_pedido INT NOT NULL,
  status_antigo VARCHAR(30),
  status_novo VARCHAR(30),
  data_atualizacao DATETIME NOT NULL
);
```

```
CREATE TABLE alertas (
  id_alerta INT PRIMARY KEY AUTO_INCREMENT,
  mensagem VARCHAR(255) NOT NULL,
  data_alerta DATETIME NOT NULL
);
```

```
CREATE TABLE clientes_premiados (
  id_premiacao INT PRIMARY KEY AUTO_INCREMENT,
  id_cliente INT NOT NULL,
  nome VARCHAR(100) NOT NULL,
  data_premiacao DATETIME NOT NULL
);
```

Explicação linha por linha: PRIMARY KEY cria o identificador único; AUTO_INCREMENT gera o número automaticamente; NOT NULL obriga preenchimento; UNIQUE evita email repetido; FOREIGN KEY liga pedidos ao cliente correto.

Print: tire print da criação das tabelas ou do comando SHOW TABLES.

```
SHOW TABLES;
```

Etapa 3 - Inserir dados iniciais

O que será feito: preencher algumas fichas para testar os comandos e triggers.

Analogia: antes de testar o sistema da loja, precisamos cadastrar alguns clientes e pedidos fictícios, como massa de teste.

```
INSERT INTO clientes (nome, email, status_cliente, tipo_cliente, data_cadastro) VALUES
('Ana Souza', 'ana@datassoft.com', 'Ativo', 'online', '2026-06-01'),
('Bruno Lima', 'bruno@datassoft.com', 'Ativo', 'online', '2026-06-02'),
('Carla Mendes', 'carla@datassoft.com', 'Ativo', 'presencial', '2026-06-03'),
('Diego Alves', 'diego@datassoft.com', 'Inativo', 'online', '2026-06-04');
```

```
INSERT INTO pedidos (id_cliente, data_pedido, status) VALUES
```



```
(1, '2026-06-01', 'Pendente'),
(1, '2026-06-02', 'Processando'),
(1, '2026-06-03', 'Cancelado'),
(2, '2026-06-04', 'Enviado'),
(3, '2026-06-05', 'Pendente'),
(3, '2026-06-06', 'Cancelado'),
(4, '2024-01-10', 'Pendente');
```

Explicação: Ana possui três pedidos; Bruno possui um pedido enviado; Carla possui dois pedidos; Diego possui um pedido antigo. Isso permite testar contagem, bloqueio e exclusão.

Print: tire print do resultado dos SELECTs abaixo.

```
SELECT * FROM clientes;
SELECT * FROM pedidos;
```

Etapa 4 - Exemplo prático de INSERT com SELECT, JOIN, GROUP BY, HAVING e NOT EXISTS

O que será feito: inserir automaticamente em clientes_premiados os clientes online com três ou mais pedidos, sem duplicar quem já foi premiado.

Analogia: é como montar uma lista de alunos premiados. Primeiro juntamos as fichas de alunos e notas, contamos quem atingiu a regra e conferimos se o aluno já não está na lista.

```
INSERT INTO clientes_premiados (id_cliente, nome, data_premiacao)
SELECT c.id_cliente, c.nome, NOW()
FROM clientes AS c
JOIN pedidos AS p ON c.id_cliente = p.id_cliente
WHERE c.tipo_cliente = 'online'
AND NOT EXISTS (
    SELECT 1
    FROM clientes_premiados AS cp
    WHERE cp.id_cliente = c.id_cliente
)
GROUP BY c.id_cliente, c.nome
HAVING COUNT(p.id_pedido) >= 3;

SELECT * FROM clientes_premiados;
```

Resultado esperado: Ana deve aparecer na tabela clientes_premiados, pois é online e tem três pedidos. Se o comando for executado novamente, NOT EXISTS evita duplicidade.

Etapa 5 - Exemplo prático de UPDATE JOIN

O que será feito: atualizar pedidos antigos de clientes inativos para o status Revisar.

Analogia: se a secretaria percebe que um aluno está inativo, algumas pendências dele precisam ser revisadas. O sistema usa a ficha do cliente para atualizar a ficha do pedido.

```
UPDATE pedidos AS p
JOIN clientes AS c ON p.id_cliente = c.id_cliente
SET p.status = 'Revisar'
WHERE c.status_cliente = 'Inativo';

SELECT * FROM pedidos;
```

Resultado esperado: o pedido do Diego deve mudar para Revisar, porque Diego está Inativo.

Etapa 6 - Exemplo prático de DELETE JOIN

O que será feito: excluir pedidos muito antigos que não estejam enviados.

Analogia: é como limpar arquivos antigos do armário, mas sem apagar documentos que ainda são importantes.

```
DELETE p
FROM pedidos AS p
JOIN clientes AS c ON p.id_cliente = c.id_cliente
WHERE p.data_pedido < CURDATE() - INTERVAL 2 YEAR
AND p.status <> 'Enviado';

SELECT * FROM pedidos;
```

Resultado esperado: pedidos antigos podem ser removidos. Se não houver pedido com mais de dois anos, nenhum registro será alterado.

Etapa 7 - Criar trigger de alerta para novo cliente

O que será feito: sempre que um cliente novo for cadastrado, o banco cria automaticamente um alerta.

Analogia: é como uma campanha interna da empresa. Quando um novo cliente chega, o sistema avisa a equipe automaticamente.

```
DELIMITER //
CREATE TRIGGER after_cliente_insert
AFTER INSERT ON clientes
FOR EACH ROW
BEGIN
    INSERT INTO alertas (mensagem, data_alerta)
    VALUES (CONCAT('Novo cliente cadastrado: ', NEW.nome), NOW());
END //
DELIMITER ;
```

Teste:

```
INSERT INTO clientes (nome, email, status_cliente, tipo_cliente)
VALUES ('Elaine Rocha', 'elaine@datassoft.com', 'Ativo', 'online');

SELECT * FROM alertas;
```

Resultado esperado: a tabela alertas deve registrar uma mensagem dizendo que Elaine foi cadastrada.

Etapa 8 - Criar trigger de auditoria de atualização de pedido

O que será feito: registrar automaticamente quando o status de um pedido for alterado.

Analogia: auditoria é como uma câmera de segurança: ela não impede a alteração, mas registra que algo aconteceu.

```
DELIMITER //
CREATE TRIGGER after_pedido_update
AFTER UPDATE ON pedidos
FOR EACH ROW
BEGIN
    IF OLD.status <> NEW.status THEN
```

```

INSERT INTO auditoria_pedidos (id_pedido, status_antigo, status_novo, data_atualizacao)
VALUES (NEW.id_pedido, OLD.status, NEW.status, NOW());
END IF;
END //
DELIMITER ;

```

Teste:

```

UPDATE pedidos
SET status = 'Processando'
WHERE id_pedido = 1;

SELECT * FROM auditoria_pedidos;

```

Resultado esperado: a tabela auditoria_pedidos deve registrar o pedido atualizado, o status antigo, o status novo e a data/hora da atualização.

Etapa 9 - Criar trigger para excluir pedidos ao remover cliente

O que será feito: quando um cliente for excluído, seus pedidos também serão removidos automaticamente, desde que não exista pedido Enviado protegido por outra regra.

Analogia: se a ficha de um cliente for removida, não faz sentido deixar pedidos órfãos perdidos no armário. A trigger funciona como um funcionário que confere e limpa as fichas relacionadas antes da ficha principal sair do arquivo.

```

DELIMITER //
CREATE TRIGGER before_cliente_delete
BEFORE DELETE ON clientes
FOR EACH ROW
BEGIN
    DELETE FROM pedidos
    WHERE id_cliente = OLD.id_cliente;
END //
DELIMITER ;

```

Teste recomendado: excluir Carla, que possui pedidos não enviados. Esse teste deve funcionar sem erro.

```

DELETE FROM clientes
WHERE id_cliente = 3;

SELECT * FROM clientes;
SELECT * FROM pedidos;

```

Resultado esperado: Carla deve sair de clientes, e os pedidos dela também devem sumir da tabela pedidos.

Print: tire print mostrando que Carla saiu da tabela clientes e que os pedidos dela não aparecem mais em pedidos.

Etapa 10 - Criar trigger para bloquear exclusão de pedido enviado

O que será feito: impedir que pedidos com status Enviado sejam apagados.

Analogia: um pedido enviado é como uma encomenda que já saiu para entrega. Não podemos simplesmente apagar o registro, porque ele é importante para rastreamento e histórico. A trigger funciona como uma trava de segurança.

```

DELIMITER //

```

```
CREATE TRIGGER before_pedido_delete
BEFORE DELETE ON pedidos
FOR EACH ROW
BEGIN
    IF OLD.status = 'Enviado' THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Não é permitido excluir pedidos enviados.';
    END IF;
END //
DELIMITER ;
```

Teste: execute este teste por último, pois ele deve gerar erro de propósito.

```
DELETE FROM pedidos
WHERE status = 'Enviado';
```

Resultado esperado: o banco deve mostrar erro com a mensagem da trigger. Esse erro é positivo, porque prova que a exclusão indevida foi bloqueada.

Print: tire print do erro e escreva abaixo: "O erro apareceu porque a trigger funcionou como trava de segurança".

Atenção: O teste de bloqueio deve ficar por último. Se o aluno executar esse DELETE no meio de um bloco grande de comandos, alguns ambientes podem parar a execução no erro esperado. Isso não significa que o código está errado; significa que a proteção funcionou.

9.1 Alternativa para quem não tem MySQL: OneCompiler com SQLite

Quando usar esta parte: Use esta versão quando os computadores da escola não tiverem MySQL, XAMPP ou Workbench funcionando. O aluno acessa o OneCompiler pelo navegador, escolhe SQLite e executa os comandos. A lógica da atividade é a mesma: criar tabelas, inserir dados, criar triggers e testar automações.

Analogia geral: se o MySQL fosse uma cozinha profissional instalada no computador, o OneCompiler seria uma cozinha emprestada pela internet. Você não instala nada; apenas entra no site, cola o código e testa. O sabor da receita é parecido, mas alguns ingredientes mudam de nome, porque no OneCompiler usaremos SQLite.

MySQL	OneCompiler/SQLite	O que muda
CREATE DATABASE e USE	Não usar	No OneCompiler, o ambiente já abre um banco temporário para teste.
AUTO_INCREMENT	AUTOINCREMENT	O nome muda, mas a ideia é a mesma: gerar id automaticamente.
NOW()	datetime('now')	Ambos pegam data e hora atuais.
CURDATE()	date('now')	Ambos pegam a data atual.
DELIMITER //	Não usar	SQLite não precisa trocar delimitador para criar trigger.
SIGNAL SQLSTATE	RAISE(ABORT, 'mensagem')	Ambos servem para bloquear uma ação e mostrar erro.

Como abrir o OneCompiler

- Acesse o site OneCompiler pelo navegador.
- Na lista de linguagens, escolha SQLite.
- Apague o código inicial, se houver.
- Cole os blocos de código desta seção na área de edição.
- Clique em Run para executar.
- Tire print do código e do resultado após cada etapa importante.

Explicação para os alunos: O OneCompiler é um laboratório online. Ele é ótimo para treinar comandos SQL sem instalar nada. O banco criado ali é temporário, como uma lousa: serve para a aula, mas pode apagar quando a página for fechada.

OneCompiler - Etapa 1: criar as tabelas

O que será feito: criar as mesmas gavetas da DataSoft, mas usando sintaxe compatível com SQLite.

Analogia: continuamos criando o armário da empresa, só que agora em uma sala online. As gavetas têm a mesma função: clientes, pedidos, auditoria, alertas e premiados.

```
PRAGMA foreign_keys = ON;

DROP TABLE IF EXISTS clientes_premiados;
DROP TABLE IF EXISTS alertas;
DROP TABLE IF EXISTS auditoria_pedidos;
DROP TABLE IF EXISTS pedidos;
DROP TABLE IF EXISTS clientes;

CREATE TABLE clientes (
    id_cliente INTEGER PRIMARY KEY AUTOINCREMENT,
    nome TEXT NOT NULL,
    email TEXT NOT NULL UNIQUE,
    status_cliente TEXT DEFAULT 'Ativo',
    tipo_cliente TEXT DEFAULT 'online',
    data_cadastro TEXT DEFAULT (date('now'))
);

CREATE TABLE pedidos (
    id_pedido INTEGER PRIMARY KEY AUTOINCREMENT,
    id_cliente INTEGER NOT NULL,
    data_pedido TEXT NOT NULL,
    status TEXT NOT NULL,
    FOREIGN KEY (id_cliente) REFERENCES clientes(id_cliente)
);

CREATE TABLE auditoria_pedidos (
    id_auditoria INTEGER PRIMARY KEY AUTOINCREMENT,
    id_pedido INTEGER NOT NULL,
    status_antigo TEXT,
    status_novo TEXT,
    data_atualizacao TEXT NOT NULL
```

```
);

CREATE TABLE alertas (
  id_alerta INTEGER PRIMARY KEY AUTOINCREMENT,
  mensagem TEXT NOT NULL,
  data_alerta TEXT NOT NULL
);

CREATE TABLE clientes_premiados (
  id_premiacao INTEGER PRIMARY KEY AUTOINCREMENT,
  id_cliente INTEGER NOT NULL,
  nome TEXT NOT NULL,
  data_premiacao TEXT NOT NULL
);
```

Explicação detalhada: PRAGMA foreign_keys = ON ativa o respeito às ligações entre tabelas; DROP TABLE limpa execuções anteriores; CREATE TABLE cria as estruturas; INTEGER PRIMARY KEY AUTOINCREMENT gera o código automaticamente; TEXT guarda textos e datas em formato simples no SQLite.

Print: tire print da execução sem erro. Se aparecer erro, verifique se escolheu SQLite, e não MySQL ou Java.

OneCompiler - Etapa 2: inserir dados iniciais

O que será feito: preencher fichas de teste para que as triggers tenham situações para reagir.

Analogia: antes de testar um alarme, precisamos ter portas e pessoas passando por elas. Antes de testar triggers, precisamos ter clientes e pedidos.

```
INSERT INTO clientes (nome, email, status_cliente, tipo_cliente, data_cadastro) VALUES
('Ana Souza', 'ana@datassoft.com', 'Ativo', 'online', '2026-06-01'),
('Bruno Lima', 'bruno@datassoft.com', 'Ativo', 'online', '2026-06-02'),
('Carla Mendes', 'carla@datassoft.com', 'Ativo', 'presencial', '2026-06-03'),
('Diego Alves', 'diego@datassoft.com', 'Inativo', 'online', '2026-06-04');

INSERT INTO pedidos (id_cliente, data_pedido, status) VALUES
(1, '2026-06-01', 'Pendente'),
(1, '2026-06-02', 'Processando'),
(1, '2026-06-03', 'Cancelado'),
(2, '2026-06-04', 'Enviado'),
(3, '2026-06-05', 'Pendente'),
(3, '2026-06-06', 'Cancelado'),
(4, '2023-01-10', 'Pendente');

SELECT * FROM clientes;
SELECT * FROM pedidos;
```

Resultado esperado: devem aparecer quatro clientes e sete pedidos. Ana terá três pedidos, Bruno terá um pedido Enviado, Carla terá dois pedidos e Diego terá um pedido antigo.

Print: tire print mostrando os SELECTs com dados.

OneCompiler - Etapa 3: INSERT com SELECT, JOIN, GROUP BY, HAVING e NOT EXISTS

O que será feito: inserir automaticamente clientes premiados, usando uma regra.

Analogia: é como criar a lista de alunos premiados a partir das notas. O sistema confere quem cumpre a regra e evita escrever o mesmo nome duas vezes.

```
INSERT INTO clientes_premiados (id_cliente, nome, data_premiacao)
SELECT c.id_cliente, c.nome, datetime('now')
FROM clientes AS c
JOIN pedidos AS p ON c.id_cliente = p.id_cliente
WHERE c.tipo_cliente = 'online'
AND NOT EXISTS (
  SELECT 1
  FROM clientes_premiados AS cp
  WHERE cp.id_cliente = c.id_cliente
)
GROUP BY c.id_cliente, c.nome
HAVING COUNT(p.id_pedido) >= 3;

SELECT * FROM clientes_premiados;
```

Resultado esperado: Ana deve aparecer em clientes_premiados. Se executar de novo, não deve duplicar Ana por causa do NOT EXISTS.

Print: tire print da tabela clientes_premiados.

OneCompiler - Etapa 4: UPDATE usando subconsulta

O que será feito: atualizar pedidos de clientes inativos para Revisar. No MySQL usamos UPDATE JOIN; no SQLite usamos subconsulta com IN.

Analogia: é como pegar a lista de clientes inativos e depois revisar todos os pedidos que pertencem a esses clientes.

```
UPDATE pedidos
SET status = 'Revisar'
WHERE id_cliente IN (
  SELECT id_cliente
  FROM clientes
  WHERE status_cliente = 'Inativo'
);

SELECT * FROM pedidos;
```

Resultado esperado: o pedido do Diego deve mudar para Revisar.

Print: tire print da tabela pedidos após a atualização.

OneCompiler - Etapa 5: DELETE usando condição

O que será feito: excluir pedidos antigos que não estejam enviados. No SQLite, usamos DELETE direto com condições.

Analogia: limpar arquivos antigos do armário, mas sem apagar documentos importantes ou enviados.

```
DELETE FROM pedidos
```

```
WHERE data_pedido < date('now', '-2 years')
AND status <> 'Enviado';
```

```
SELECT * FROM pedidos;
```

Resultado esperado: pedidos antigos que não estejam enviados podem ser removidos. Se o pedido do Diego ainda existir e for antigo, ele será removido.

Print: tire print do resultado após o DELETE.

OneCompiler - Etapa 6: trigger de alerta de novo cliente

O que será feito: criar um alerta automático quando um novo cliente for cadastrado.

Analogia: é uma campainha da empresa. Entrou cliente novo, o quadro de avisos recebe uma mensagem.

```
CREATE TRIGGER after_cliente_insert
AFTER INSERT ON clientes
FOR EACH ROW
BEGIN
    INSERT INTO alertas (mensagem, data_alerta)
    VALUES ('Novo cliente cadastrado: ' || NEW.nome, datetime('now'));
END;

INSERT INTO clientes (nome, email, status_cliente, tipo_cliente)
VALUES ('Elaine Rocha', 'elaine@datassoft.com', 'Ativo', 'online');

SELECT * FROM alertas;
```

Resultado esperado: a tabela alertas deve mostrar uma mensagem para Elaine Rocha. No SQLite, usamos || para juntar textos, em vez de CONCAT().

Print: tire print da trigger e do SELECT em alertas.

OneCompiler - Etapa 7: trigger de auditoria de atualização

O que será feito: registrar automaticamente a mudança de status de um pedido.

Analogia: câmera de segurança do banco. Toda alteração de status fica registrada para conferência.

```
CREATE TRIGGER after_pedido_update
AFTER UPDATE ON pedidos
FOR EACH ROW
WHEN OLD.status <> NEW.status
BEGIN
    INSERT INTO auditoria_pedidos (id_pedido, status_antigo, status_novo, data_atualizacao)
    VALUES (NEW.id_pedido, OLD.status, NEW.status, datetime('now'));
END;

UPDATE pedidos
SET status = 'Processando'
WHERE id_pedido = 1;

SELECT * FROM auditoria_pedidos;
```

Resultado esperado: auditoria_pedidos deve registrar o id do pedido, o status antigo, o novo e a data/hora.

Print: tire print da tabela auditoria_pedidos.

OneCompiler - Etapa 8: trigger para excluir pedidos ao remover cliente

O que será feito: excluir os pedidos ligados ao cliente antes de remover o cliente.

Analogia: antes de jogar fora a ficha de um cliente, o funcionário automático recolhe os pedidos ligados a ele para não deixar documentos soltos.

```
CREATE TRIGGER before_cliente_delete
BEFORE DELETE ON clientes
FOR EACH ROW
BEGIN
    DELETE FROM pedidos
    WHERE id_cliente = OLD.id_cliente;
END;

DELETE FROM clientes
WHERE id_cliente = 3;

SELECT * FROM clientes;
SELECT * FROM pedidos;
```

Resultado esperado: Carla deve sair da tabela clientes, e os pedidos dela também devem sair da tabela pedidos.

Print: tire print do resultado final das tabelas clientes e pedidos.

OneCompiler - Etapa 9: trigger para bloquear exclusão de pedido enviado

O que será feito: impedir que pedido com status Enviado seja apagado.

Analogia: é uma trava de segurança. Se a encomenda já saiu para entrega, ninguém pode apagar o registro sem controle.

```
CREATE TRIGGER before_pedido_delete
BEFORE DELETE ON pedidos
FOR EACH ROW
WHEN OLD.status = 'Enviado'
BEGIN
    SELECT RAISE(ABORT, 'Não é permitido excluir pedidos enviados.');
```

```
END;

DELETE FROM pedidos
WHERE status = 'Enviado';
```

Resultado esperado: o OneCompiler deve mostrar erro com a mensagem da trigger. Esse erro é positivo, porque prova que o bloqueio funcionou.

Print: tire print do erro. Explique abaixo: "o erro aconteceu porque a trigger impediu a exclusão".

Importante no OneCompiler: Deixe o teste do pedido Enviado por último. Como ele gera erro de propósito, o OneCompiler pode interromper a execução naquele ponto. Isso é esperado: é a trava de segurança funcionando.

Cuidado no OneCompiler: Se você executar o mesmo código duas vezes sem apagar as triggers ou reiniciar o ambiente, pode aparecer erro dizendo que a trigger já existe. Isso é como tentar instalar o

mesmo alarme duas vezes. Para evitar, rode tudo do começo em um ambiente limpo ou use os comandos DROP TABLE no início.

Resumo para o professor: MySQL x OneCompiler

- Mantenha a versão MySQL para alunos que tiverem ambiente local funcionando.
- Use a versão OneCompiler/SQLite para alunos sem permissão de instalação ou sem MySQL nas máquinas.
- Explique que a lógica é a mesma, mas a sintaxe muda em alguns pontos.
- No MySQL, usamos DELIMITER, NOW(), CURDATE(), CONCAT() e SIGNAL SQLSTATE.
- No SQLite/OneCompiler, usamos datetime('now'), date('now'), || para concatenar e RAISE(ABORT, mensagem).
- O objetivo pedagógico é o mesmo: entender tabelas relacionadas, comandos DML e triggers.

10. Prints e evidências obrigatórias

- Escolha do ambiente: MySQL ou OneCompiler/SQLite.
- Criação do banco de dados no MySQL ou criação das tabelas no OneCompiler.
- Criação das tabelas e comando SHOW TABLES no MySQL, ou SELECTs de conferência no OneCompiler.
- Inserção dos dados iniciais.
- SELECT inicial de clientes e pedidos.
- Execução do exemplo INSERT INTO ... SELECT ... JOIN.
- Resultado de clientes_premiados.
- Execução do UPDATE JOIN no MySQL ou UPDATE com subconsulta no OneCompiler, e resultado em pedidos.
- Execução do DELETE JOIN no MySQL ou DELETE com condição no OneCompiler, e resultado em pedidos.
- Criação da trigger after_cliente_insert e teste na tabela alertas.
- Criação da trigger after_pedido_update e teste na auditoria.
- Criação da trigger before_cliente_delete e teste de exclusão de cliente.
- Criação da trigger before_pedido_delete e erro ao tentar excluir pedido Enviado.
- Resultado final das tabelas clientes, pedidos, auditoria_pedidos e alertas.

11. Organização no Word/Docs e Excel/Sheets

11.1 Word ou Google Docs

- Título da atividade.
- Nome dos integrantes.
- Objetivo da prática.
- Prints organizados em ordem.
- Explicação curta abaixo de cada print: o que foi feito, qual comando foi usado, o que aconteceu e qual conceito foi demonstrado.
- Conclusão final explicando o que uma trigger faz e por que ela ajuda na automação do banco.

11.2 Excel ou Google Sheets

Etapa	Comando utilizado	O que o comando faz	Analogia	Resultado esperado	Resultado obtido	Print?
Criar banco	CREATE DATABASE	Cria o banco DataSoft	Cria uma pasta principal	Banco criado		Sim/Não

Criar tabelas	CREATE TABLE	Cria estruturas	Cria gavetas do armário	Tabelas criadas		Sim/Não
Inserir dados	INSERT INTO	Preenche registros	Preenche fichas	Dados inseridos		Sim/Não
Auditoria	CREATE TRIGGER	Registra atualização	Câmera de segurança	Auditoria gerada		Sim/Não

12. Se aparecer erro

Erro comum	O que pode significar	Como explicar com analogia
Erro de sintaxe	Algum comando foi digitado com palavra, vírgula, parêntese ou ponto e vírgula incorreto.	É como uma frase com pontuação errada: o banco não entende a ordem.
Table already exists	A tabela já foi criada anteriormente.	É como tentar criar uma gaveta que já existe no armário.
Trigger already exists	A trigger já foi criada.	É como tentar instalar o mesmo alarme duas vezes.
Cannot delete or update a parent row	Existe relação entre tabelas impedindo uma exclusão insegura.	É como tentar jogar fora a ficha do cliente enquanto ainda existem documentos ligados a ele.
Não é permitido excluir pedidos enviados	A trigger funcionou corretamente e bloqueou a exclusão.	A trava de segurança impediu apagar uma encomenda que já saiu para entrega.
Unknown column	Nome de campo digitado errado ou campo não existe.	É como procurar uma coluna que não existe na planilha.
Erro ao usar DELIMITER no OneCompiler	SQLite não usa DELIMITER.	É como tentar usar uma regra de uma cozinha em outra cozinha diferente.
Erro ao usar CONCAT no OneCompiler	SQLite junta textos com , não com CONCAT().	A ideia é juntar nomes, mas a ferramenta usa outra fita adesiva.

13. Perguntas de fixação

- Qual é a diferença entre DDL e DML? Dê um exemplo de cada.

Resposta: DDL é usada para criar e modificar a estrutura do banco de dados, enquanto a DML é usada para manipular os dados armazenados nele

Exemplo de DDL: criar uma tabela de clientes no banco de dados.

Exemplo de DML: adicionar um novo cliente na tabela.

- Por que uma chave primária é importante em uma tabela?

Resposta: é importante porque identifica cada registro de forma única dentro de uma tabela, evitando que existam dados duplicados.

- Para que serve uma chave estrangeira?

Resposta: serve para relacionar duas tabelas

- O que o JOIN permite fazer?

Resposta: permite combinar dados de duas ou mais tabelas relacionadas em uma única consulta.

5. Quando usamos GROUP BY e HAVING?

Resposta: GROUP BY é usado para agrupar registros com valores iguais, e o HAVING é usado para filtrar os grupos criados.

6. Por que o NOT EXISTS ajuda a evitar dados duplicados?

Resposta: ajuda a evitar dados duplicados porque verifica se um registro já existe antes de inserir ou selecionar informações.

7. O que uma trigger faz?

Resposta: uma rotina automática executada quando ocorre uma ação específica em uma tabela, como inserir, atualizar ou excluir dados.

8. Por que registrar auditoria de alterações pode ser importante?

Resposta: Registrar auditoria de alterações é importante para acompanhar quem fez mudanças, quando elas ocorreram e quais dados foram modificados.

9. Por que bloquear a exclusão de pedidos enviados é uma boa prática?

Resposta: Bloquear a exclusão de pedidos enviados é uma boa prática porque evita a perda de informações importantes e mantém o histórico das operações.

10. Explique com suas palavras a analogia da trigger como alarme automático.

Resposta: A trigger pode ser comparada a um alarme automático porque ela é acionada sozinha quando um evento específico acontece.

14. Critérios de avaliação

Critério	Peso sugerido	O que será observado
Participação e organização	1,0	Trabalho em dupla/grupo, atenção aos passos e colaboração.
Compreensão teórica	2,0	Diferença entre DDL e DML, relação entre tabelas, chaves, JOIN e triggers.
Execução dos comandos SQL	2,0	Criação da base, tabelas, inserts, consultas e comandos avançados.
Triggers funcionando	2,0	Auditoria, alerta, bloqueio e exclusão automatizada testados corretamente.
Evidências	1,5	Prints organizados no Word/Docs com explicações.
Planilha de acompanhamento	1,0	Tabela no Excel/Sheets preenchida com resultados e observações.
Conclusão final	0,5	Explicação clara do que foi aprendido.

15. Fechamento da aula

Ao final da aula, reforce que DDL organiza a estrutura do banco, DML manipula os dados, JOIN conecta informações entre tabelas e triggers automatizam ações importantes. A prática mostra que o banco de dados pode agir como um sistema inteligente de organização: ele registra, protege e automatiza tarefas para reduzir erros humanos.

Frase final para os alunos: Hoje vocês não apenas executaram comandos SQL. Vocês criaram uma pequena regra de negócio dentro do banco de dados, fazendo o sistema reagir sozinho a determinados acontecimentos.

16. Anexo - sequência resumida de comandos

Use este anexo apenas como checklist rápido. A explicação detalhada está nas etapas anteriores.

```

DROP DATABASE IF EXISTS datassoft;
CREATE DATABASE datassoft;
USE datassoft;

-- Criar tabelas: clientes, pedidos, auditoria_pedidos, alertas, clientes_premiados
-- Inserir dados iniciais
-- Consultar SELECT * FROM clientes; SELECT * FROM pedidos;
-- Testar INSERT INTO ... SELECT ... JOIN com GROUP BY, HAVING e NOT EXISTS
-- Testar UPDATE JOIN
-- Testar DELETE JOIN
-- Criar triggers:
-- 1) after_cliente_insert
-- 2) after_pedido_update
-- 3) before_cliente_delete
-- 4) before_pedido_delete
-- Testar cada trigger e registrar prints

```